EPHEMERAL

Hardening exp022 for Wilkes3

ar065 · 11 August 2026 · Draft

Exp022 trains the shared TR-01–TR-06 checkpoint bank. The run will use Cambridge Wilkes3 under the SL2 Slurm allocation. This checklist covers the work needed to avoid wasting a large allocation or losing results. It is intentionally scoped to one research campaign, not a general workflow system.

Check an item only after it has been exercised. A stage is complete when all its items and its exit criterion pass.

Stage 1: confirm what will run

- ☑ **Complete** `H1.1` `experiments/exp022.py` contains one registry for cell names, TR IDs, parameters, seeds, and resource tiers.
- ☑ **Complete** `H1.2` `--train-cell NAME` and `--list-cells TIER` use that registry, so the Slurm wrapper does not maintain a separate experiment definition.
- ☑ **Complete** `H1.3` TR-06 defines three variable-rate PING cells using the `spike-rate` output-LIF readout required by exp082.
- ☑ **Complete** `H1.4` Add a concise campaign manifest command that records the git commit, complete cell list, TR IDs, parameters, resource tiers, and output root.
- ☑ **Complete** `H1.5` Add tests that cell names are unique, all cells have one resource tier, and generated commands contain the expected scientific parameters.

Exit criterion. We can print and review the exact cells and commands that Wilkes3 will execute.

Stage 2: keep runs separate and recoverable

- ☑ **Complete** `H2.1` Require an explicit output directory for HPC training. Put each smoke test or production campaign in its own named directory.
- ☑ **Complete** `H2.2` Refuse to overwrite a completed cell by default. A repeated submission should skip valid cells and rerun only missing or failed ones.
- ☑ **Complete** `H2.3` Define a simple completion check: required configuration, metrics history, and loadable checkpoint must all exist and match the requested cell.

- ☑ **Complete** `H2.4` Write the git commit, command, Slurm job and task IDs, host, and GPU into each cell's run record.

- ☑ **Complete** `H2.5` Keep partial failed output distinguishable from completed output. Restarting a cell from scratch is acceptable; epoch-level resume is not required.

Exit criterion. A smoke test cannot overwrite production data, and resubmitting the campaign safely fills gaps.

Stage 3: prepare Wilkes3

- ❓ **Incomplete** `H3.1` Document the checkout, module, `uv`, and environment setup used on an Ampere compute node. Pin the production run to a reviewed commit and lockfile.

- ❓ **Incomplete** `H3.2` Populate MNIST in a persistent shared cache before the campaign. Confirm compute nodes can use it without competing downloads.

- ❓ **Incomplete** `H3.3` Choose persistent locations for the repository, environment, dataset cache, campaign outputs, and logs. Check quota and free space.

- ❓ **Incomplete** `H3.4` Run a short Slurm diagnostic that imports the project, reports PyTorch and CUDA, sees the allocated A100, reads MNIST, and writes to the campaign directory.

Exit criterion. A non-interactive GPU job can start the pinned code, read the data, and write results successfully.

Stage 4: smoke test and measure resources

- ❓ **Incomplete** `H4.1` Run the local test suite and a tiny plumbing run in a new disposable output directory.

- ❓ **Incomplete** `H4.2` Run the same plumbing test through Slurm and inspect its configuration, metrics, checkpoint, logs, and provenance.

- ❓ **Incomplete** `H4.3` Run one representative canary from each resource tier: standard, fine timestep, canonical COBA, canonical PING, and variable rate.

- ❓ **Incomplete** `H4.4` Record wall time and peak memory from the canaries, then replace provisional Slurm requests with sensible margins.

- ❓ **Incomplete** `H4.5` Interrupt or fail one disposable cell and confirm that resubmission reruns it without touching completed cells.

Exit criterion. Every job shape has run successfully on Wilkes3 and the final resource requests are based on measurements.

Stage 5: submit and monitor the campaign

- ☑ **Complete** `H5.1` The current Slurm scaffold maps one array task to one registry cell and requests one Ampere GPU per task.
- ☑ **Complete** `H5.2` Update the submission wrapper to require the account, campaign output directory, and tier. Print the resolved cells, wall time, concurrency, and destination before submission.
- ❓ **Incomplete** `H5.3` Run `sbatch --test-only`, check the SL2 balance and queue, then submit the measured tiers. Record returned job IDs with the campaign.
- ☑ **Complete** `H5.4` Provide a compact status command or script showing completed, running, failed, and missing cells. Slurm completion alone does not count as a valid trained cell.
- ❓ **Incomplete** `H5.5` Retry only failed or missing cells after checking the cause. Do not change scientific parameters within the same campaign.

Exit criterion. Every planned cell is complete and loadable, or any omission is explicitly recorded.

Stage 6: aggregate, inspect, and archive

- ☑ **Complete** `H6.1` Exp022 already has an aggregation path that writes `numbers.json`, training curves, and representative rasters from the shared cell bank.
- ❓ **Incomplete** `H6.2` Aggregate only after checking that every expected cell is present and valid. Produce one training curve and representative raster for each TR ID.
- ❓ **Incomplete** `H6.3` Load representative checkpoints independently and run an exp082 smoke test against all three TR-06 cells.
- ❓ **Incomplete** `H6.4` Inspect exp022 in the Demolab web UI, then deliberately promote the verified aggregate into `artifacts/data/exp022`.
- ❓ **Incomplete** `H6.5` Archive the expensive checkpoint bank to R2, run an R2/local check, and record the snapshot manifest and restore command.

Exit criterion. Exp022 builds from a verified checkpoint bank, exp082 can consume TR-06, and the training data has a checked remote backup.

Ready to launch

The production arrays are ready when Stages 1–4 are complete. Stages 5 and 6 then guide the live campaign and its handoff to downstream experiments. The essential safeguards are simple: reviewed commands, explicit output paths, non-overwriting retries, a tested Wilkes3 environment, measured resource requests, loadable checkpoints, and an R2 backup.

Timestamped evidence

2026-08-11T15:34:36Z — implementation checkpoint. Commit `cdd17980` introduced the registry-backed manifest, explicit campaign layout, cell validator, non-overwriting retry path, atomic per-cell attempt records, status output, hardened Slurm wrappers, Wilkes3 diagnostic, and operator runbook. Focused registry, validator, retry, and exp082 compatibility checks passed: 20 tests.

2026-08-11T15:26:47Z — local manifest rehearsal. A clean-commit disposable TR-06 plumbing manifest validated its hash and registry identity. Status reported three missing cells. The dry-run wrapper printed the exact three-cell retry set, destination, tier, wall time, concurrency, account placeholder, partition, array range, and `sbatch` command without submitting work.

2026-08-11T15:30:11Z — preserved host-limit failure. The registered 100-sample, two-epoch variable-rate PING plumbing cell reached model construction and compilation, then the 4 GB host killed it with exit code −9. The campaign retained its configuration, empty history, logs, and sanitized attempt record and classified the cell as failed rather than complete. No scientific parameter was changed.

2026-08-11T15:31:08Z — retry rehearsal. Retrying the failed cell first moved the prior partial directory into the campaign's timestamped `failed/` area. The replacement attempt was deliberately interrupted and recorded as failed. A focused test separately confirmed that the same retry entry point does not launch training or alter the checkpoint when a cell validates as complete.

2026-08-11T16:07:30Z — safety-review remediation. Commit `8038bffd` added atomic per-cell ownership, authoritative running status, retry exclusion for active and stale attempts, explicit stale recovery after a positive inactivity check, and collision-safe failed-output preservation. Manifest validation now reconstructs and checks the complete cell selection, command, shell rendering, output path, required files, registry parameters, tool path, lockfile, and campaign-root relationship before executing a freshly reconstructed command. External campaign roots are recorded explicitly, post-aggregation checks permit only exp022′s generated artifacts, and aggregation still requires every cell to validate. The R2 path now selects the exact verified campaign bank and records campaign and per-file hash provenance; restore requires a separate destination. The MNIST link helper tolerates concurrent creation while rejecting a different target. Ruff, focused type checking, shell syntax, and 36 focused campaign/exp082 tests passed. No training, Slurm diagnostic, canary, production array, aggregation, or R2 transfer was run in this remediation, so Stages 3, 4, and 6 remain operationally incomplete.

Operational boundary. This host has no configured Wilkes3 login, `sbatch`, or `mybalance`. Stages 3 and 4 therefore remain operationally incomplete; no diagnostic, Slurm plumbing job, canary, or production array has been submitted.